2. Linear Models for Regression

Gholamreza Haffari

2. Linear Models for Regression

Gholamreza Haffari

Generated by Alexandria (https://www.alexandriarepository.org) on January 9, 2019 at 4:37 pm AEDT

COPYRIGHT WARNING

This ebook is protected by copyright. For use within Monash University only. NOT FOR RESALE Do not remove this notice.

Disclaimer:
https://www.monash.edu/disclaimer-copyright

Contents

Title i
Copyright ii
1 Linear Basis Function Models 1
2 Learning Model Parameters 3
3 Activity 2.1 Linear Regression 8
4 Regularisation 9
5 Activity 2.2 Regularization 13
6 Bias-Variance Analysis 14
7 Activity 2.3 Bias-Variance in Regression 20

1
Linear Basis Function Models

The goal of *regression* is to predict the value of one or more *continuous* target variables t given the value of a d-dimensional vector $\mathbf{x}$ of input variables. We have already encountered an example of a regression problem when we considered polynomial curve fitting in Module 1 (https://www.alexandriarepository.org/reader-preview/84659/91098). The polynomial is a specific example of a broad class of functions called *linear regression models*, which share the property of being linear functions of the adjustable parameters, and which will form the focus of this chapter.

The simplest form of linear regression models are also linear functions of the input variables. However, we can obtain a much more useful class of functions by taking linear combinations of a fixed set of *nonlinear* functions of the input variables, known as *basis functions*. Such models are linear functions of the parameters, which gives them simple analytical properties, and yet can be nonlinear with respect to the input variables.

Linear Basis Functions. A function $f(\cdot)$ is called linear if and only if

$$f(\alpha\mathbf{x}_1 + \beta\mathbf{x}_2) = \alpha f(\mathbf{x}_1) + \beta f(\mathbf{x}_2) \qquad \forall \mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^d \quad \forall \alpha, \beta \in \mathbb{R}$$

Linear functions will lead to simple learning algorithms, so let's see what we can do with them. The simplest linear model for regression is:

$$y(\mathbf{x}, \mathbf{w}) := w_0 + w_1 x_1 + w_2 x_2 + \ldots + w_d x_d$$

Remember that we are learning $\mathbf{w}$, i.e. we would like to set $\mathbf{w}$ so that $y(\mathbf{x}, \mathbf{w})$ aligns with target value in training data.

Non-Linear Basis Functions. We extend our linear models to include fixed non-linear functions of *data:*

$$\begin{aligned} y(\mathbf{x}, \mathbf{w}) &:= w_0 + w_1\phi_1(\mathbf{x}) + w_2\phi_2(\mathbf{x}) + \ldots + w_{M-1}\phi_{M-1}(\mathbf{x}) \\ &= \sum_{j=0}^{M-1} w_j \phi_j(\mathbf{x}) \\ &= \mathbf{w} \cdot \boldsymbol{\phi}(\mathbf{x}) \end{aligned}$$

where we define $\phi_0(\mathbf{x}) := 1$ to be a dummy basis function, $\mathbf{w} := (w_0, \ldots, w_{M-1})$ to be the parameters, and $\boldsymbol{\phi}(\mathbf{x}) = (\phi_0(\mathbf{x}), \ldots, \phi_{M-1}(\mathbf{x}))$ to be the basis function vector. By using *nonlinear basis functions* $\phi_j(\cdot)$, we allow the function $y(\mathbf{x}, \mathbf{w})$ to be a non-linear function of the *input*. Functions of the above form are called *linear models*, however, because this function is linear in *parameters* $\mathbf{w}$. In essence, it is this linearity in the parameters that greatly simplifies the analysis of this class of models, and allows simple learning algorithms to tune the parameters.

Popular basis Functions. There are many other possible choices for the basis functions, for example

$$\phi_j(x) := \exp\left(-\frac{(x-\mu_j)^2}{2s^2}\right)$$

where μ_j governs the locations of the basis functions in input space, and the parameters govern their spatial scale. These are usually referred to as *Gaussian* basis functions, although it should be noted that they are not required to have a probabilistic interpretation, and in particular the normalization coefficient is unimportant because these basis functions will be multiplied by adaptive parameters w_j .

Another possibility is the sigmoidal basis function of the form

$$\phi_j(x) := \sigma(\frac{x-\mu_j}{s})$$

where $\sigma(a)$ is the logistic sigmoid function defined by

$$\sigma(a) := \frac{1}{1+\exp(-a)}$$

Equivalently, we can use the hyperbolic tangent function $tanh(a) = \frac{1-e^{-2a}}{1+e^{-2a}}$, because it is related to the logistic sigmoid by $tanh(a) := 2\sigma(2a) - 1$. We also know that a general linear combination of logistic sigmoid functions is equivalent to a general linear combination of $tanh$ functions. These various choices of basis function are illustrated in Figure **2.1.1.**

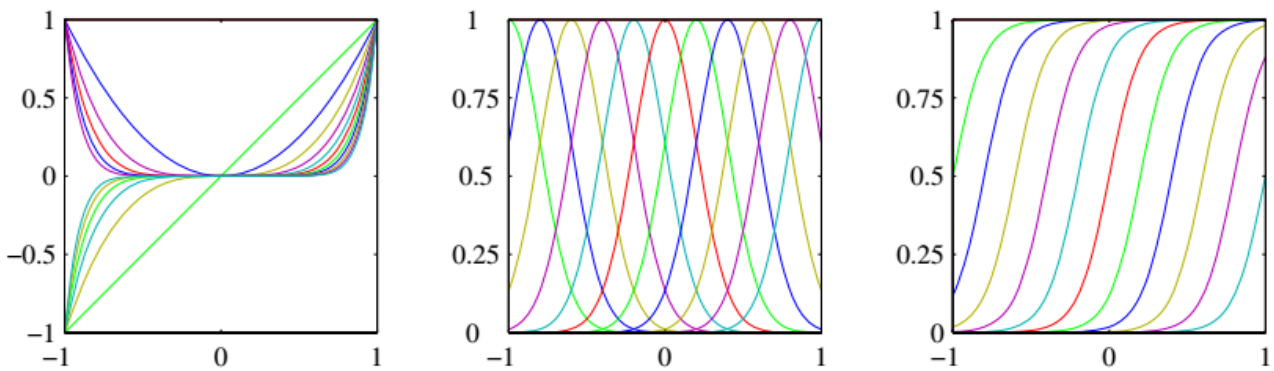

Figure 2.1.1 Examples of basis functions, showing polynomials on the left, Gaussians of in the centre, and sigmoidal on the right.

2
Learning Model Parameters

Loss Function for Regression

In Module 1, we fitted polynomial functions to data sets by minimising a sum-of-squares error function. Here we show that this is rigorous choice of the error function by making a connection to maximum likelihood principle. We show that the sum-of-squares error function can be motivated as the maximum likelihood solution under an assumed Gaussian noise model.

Let us assume that we are provided with a training set of size N: $\mathcal{D} := \{(\mathbf{x}_n, t_n)\}_{n=1}^N$. Let's further assume that t arises from a deterministic function $y(\mathbf{x}, \mathbf{w})$ plus a noise ϵ:

$$t = y(\mathbf{x}, \mathbf{w}) + \epsilon$$

where the noise is generated from a Gaussian distribution with the mean zero and variance σ^2. Then, we can write

$$p(t|\mathbf{x}, \mathbf{w}, \sigma^2) = \mathcal{N}(t|y(\mathbf{x}, \mathbf{w}), \sigma^2)$$

where $\mathcal{N}(x|\mu, \sigma^2)$ denotes the probability density function of normal (Gaussian) distribution with mean (and mod and median) μ and variance σ^2.

Making the assumption that the training data points are drawn independently from the above distribution, we obtain the following expression for the likelihood function:

$$p(t_1, \ldots, t_N|\mathbf{x}_1, \ldots, \mathbf{x}_N, \mathbf{w}, \sigma) = \prod_{n=1}^N \mathcal{N}(t_n|y(\mathbf{x}_n, \mathbf{w}), \sigma^2)$$

which is a function of the adjustable parameters $\mathbf{w}$ and σ. The log-likelihood is:

$$\begin{aligned}
\mathcal{L}(\mathbf{w}) &:= \log p(t_1, \ldots, t_N|\mathbf{x}_1, \ldots, \mathbf{x}_N, \mathbf{w}, \sigma) \\
&= \log \prod_{n=1}^N \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{1}{2\sigma^2}[t_n - \mathbf{w} \cdot \boldsymbol{\phi}(\mathbf{x}_n)]^2\right) \\
&= \underbrace{-N \log \sigma - \frac{N}{2}\ln(2\pi)}_{constant\ wrt\ \mathbf{w}} - \underbrace{\frac{1}{2\sigma^2}\sum_{n=1}^N [t_n - \mathbf{w} \cdot \boldsymbol{\phi}(\mathbf{x}_n)]^2}_{squared\ error}
\end{aligned}$$

Having written down the likelihood function, we can use maximum likelihood to determine the parameters. Consider maximisation with respect to $\mathbf{w}$, which is equivalent to minimising the second term

(since the first term is constant wrt the parameters):

$$\frac{1}{2}\sum_{n=1}^{N}[t_n - \mathbf{w}\cdot\boldsymbol{\phi}(\mathbf{x}_n)]^2$$

where we have omitted σ since we are interested into the terms involving $\mathbf{w}$. The above function is exactly the sum-of-squares error function that we saw before! Therefore, we see that *maximisation of the likelihood function under a Gaussian noise distribution for a linear model is equivalent to minimising a sum-of-squares error function.*

Finding Optimal Weights

Now that we have the training objective, we need to optimise it to get the best parameter values. To optimise the training objective, we know from calculus that we need to set the partial derivatives with respect to all parameters to zero:

$$\frac{\partial}{\partial w_i}\mathcal{L}(\mathbf{w}) = \frac{1}{\sigma^2}\sum_{n=1}^{N}(t_n - \mathbf{w}\cdot\boldsymbol{\phi}(\mathbf{x}_n))\phi_i(\mathbf{x}_n) = 0 \qquad \forall i$$

Recall from calculus that the *vector* of partial derivatives with respect to all parameters is called the *gradient,* and is denoted by $\nabla\mathcal{L}(\mathbf{w})$. So In *vector* form, we need to set the gradient to zero to find the optimal parameters:

$$\nabla\mathcal{L}(\mathbf{w}) = \begin{bmatrix} \frac{\partial\mathcal{L}(\mathbf{w})}{\partial w_0} \\ \vdots \\ \frac{\partial\mathcal{L}(\mathbf{w})}{\partial w_{M-1}} \end{bmatrix} = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}$$

If we can solve the above *system* of equations, then, we have found the solution; otherwise we need to resort to *iterative numerical algorithms* which we refer to as *sequential algorithms*.

Optional Reading: Algebraic Methods for Optimising the Loss Function

Setting the gradient to zero, we get a linear system of equations; therefore, we can solve the system by matrix operations. Specifically, we get the following after setting the gradient to zero:

$$\sum_{n=1}^{N} t_n \boldsymbol{\phi}(\mathbf{x}_n)^T - \mathbf{w}^T \left(\sum_{n=1}^{N} \boldsymbol{\phi}(\mathbf{x}_n)\boldsymbol{\phi}(\mathbf{x}_n)^T \right) = \mathbf{0}^T$$
$$\Rightarrow \sum_{n=1}^{N} t_n \boldsymbol{\phi}(\mathbf{x}_n) - \left(\sum_{n=1}^{N} \boldsymbol{\phi}(\mathbf{x}_n)\boldsymbol{\phi}(\mathbf{x}_n)^T \right)^T \mathbf{w} = \mathbf{0}$$
$$\Rightarrow \left(\sum_{n=1}^{N} \boldsymbol{\phi}(\mathbf{x}_n)\boldsymbol{\phi}(\mathbf{x}_n)^T \right)^T \mathbf{w} = \sum_{n=1}^{N} t_n \boldsymbol{\phi}(\mathbf{x}_n)$$

where $\mathbf{A}^T$ is the *transpose* of the matrix or vector $\mathbf{A}$. Let's denote the target values for all training data points by the vector $\mathbf{t}$, and the basis function representation of all training data points by the matrix $\boldsymbol{\Phi}$defined as

$$\boldsymbol{\Phi} = \begin{pmatrix} \phi_0(\boldsymbol{x}_1) & \phi_1(\boldsymbol{x}_1) & \dots & \phi_{M-1}(\boldsymbol{x}_1) \\ \phi_0(\boldsymbol{x}_2) & \phi_1(\boldsymbol{x}_2) & \dots & \phi_{M-1}(\boldsymbol{x}_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_0(\boldsymbol{x}_N) & \phi_1(\boldsymbol{x}_N) & \dots & \phi_{M-1}(\boldsymbol{x}_N) \end{pmatrix}$$

Then it's easy to see the followings:

$$\sum_{n=1}^{N} t_n \boldsymbol{\phi}(\mathbf{x}_n) = \boldsymbol{\Phi}^T \mathbf{t}$$
$$\sum_{n=1}^{N} \boldsymbol{\phi}(\mathbf{x}_n)\boldsymbol{\phi}(\mathbf{x}_n)^T = \boldsymbol{\Phi}\boldsymbol{\Phi}^T$$

Therefore, we can solve our linear system of equation as follows:

$$\left(\boldsymbol{\Phi}^T\boldsymbol{\Phi}\right)\boldsymbol{w} = \boldsymbol{\Phi}^T\mathbf{t} \Rightarrow \boldsymbol{w} = \left(\boldsymbol{\Phi}^T\boldsymbol{\Phi}\right)^{-1}\boldsymbol{\Phi}^T\mathbf{t}$$

where $\mathbf{A}^{-1}$ denotes the inverse of the matrix $\mathbf{A}$. So to sum up, all we need to do for finding the optimal parameter is to construct the vector $\boldsymbol{t}$ and the matrix $\boldsymbol{\Phi}$ and then combine them as above to get the optimiser of our training objective. *This is exactly why linear basis function models are attractive: they result in a training objective which we can solve to find the optimum solution using matrix operations.*

Iterative Algorithms for Optimising the Loss Function

The Gradient Descent (GD) Algorithm. *Gradient descent* is an algorithm that minimizes functions.

Given a function defined by a set of parameters, gradient descent starts with an initial set of parameter values and iteratively moves toward a set of parameter values that minimize the function. This iterative minimization is achieved using calculus, taking steps in the negative direction of the function gradient.

Gradient descent is based on the observation that if the objective function $E(\mathbf{w})$ is differentiable in a neighborhood of a point $\mathbf{a}$, then $E(\mathbf{w})$ decreases fastest if we move from $\mathbf{a}$ in the *direction of the negative gradient* of E at $\mathbf{a}$, that is in the direction of $-\nabla E(\mathbf{a})$. Therefore, it follows that, if

$$\mathbf{b} := \mathbf{a} - \eta \nabla E(\mathbf{a})$$

for η small enough, then the new parameter $\mathbf{b}$ has a lower objective value, namely $E(\mathbf{b}) < E(\mathbf{a})$. We can turn this updating procedure to an optimisation algorithm, called the gradient descent algorithm, for minimising objective functions:

- Initialise the parameters to $\mathbf{w}^{(0)}$ and $t = 1$
- while a stopping condition is *not* met do:
  - $\mathbf{w}^{(t)} := \mathbf{w}^{(t-1)} - \eta \nabla E(\mathbf{w}^{(t-1)})$
  - $t = t + 1$

where η is called the step-size or the learning rate. The learning rate is very important, and its proper value may be different in various iterations. Therefore, we add an inner loop to find the proper value for η adaptively for different iterations:

- Initialise the parameters to $\mathbf{w}^{(0)}$ and $t = 1$
- while a stopping condition is not met do:
  - $\eta' = \eta$
  - while $\eta' > \epsilon$ do
    - $\mathbf{w} := \mathbf{w}^{(t-1)} - \eta' \nabla E(\mathbf{w}^{(t-1)})$
    - if $E(\mathbf{w}) < E(\mathbf{w^{(t-1)}})$ then **break**
    - $\eta' = \eta'/2$
  - $\mathbf{w}^{(t)} := \mathbf{w}$
  - $t = t + 1$

where η and ϵ are the parameters of the gradient descent algorithm. The inner loop, which is also called the *line search*, uses the initial learning rate η_0 to update the parameters, and halves it if it is too big iteratively (i.e. in case the error function increases instead of decreasing).

In the case of linear regression, the error function to minimise is:

$$E(\mathbf{w}) := \frac{1}{2} \sum_{n=1}^{N} [t_n - \mathbf{w} \cdot \boldsymbol{\phi}(\mathbf{x}_n)]^2$$

where $\mathcal{D} := \{(\mathbf{x}_n, t_n)\}_{n=1}^{N}$ is the training data. The gradient of the training objective is:

$$\nabla E(\mathbf{w}) = -\sum_{n=1}^{N}[t_n - \mathbf{w}\cdot\boldsymbol{\phi}(\mathbf{x}_n)]\boldsymbol{\phi}(\mathbf{x}_n)$$

which can be plugged into the gradient descent algorithm to adapt it to learn the linear regression model.

The *Stochastic* Gradient Descent (SGD) Algorithm. *Batch* techniques, such as the maximum likelihood solution discussed above, which involve processing the entire training set in one go, can be computationally costly for large data sets. Furthermore, we may not have access to all of the training data points and they may given to us one at a time, e.g real- time applications in which the data observations are arriving in a continuous stream, and predictions must be made before all of the data points are seen. Therefore, it may be worthwhile to use *sequential algorithms*, also known as *on-line* algorithms, in which the model parameters updated after seeing each individual datapoint.

We can obtain a sequential learning algorithm by applying the technique of *stochastic gradient descent*, also known as *sequential gradient descent*, as follows. If the error function comprises a sum over data points $E(\mathbf{w}) := \sum_n E_n(\mathbf{w})$ where $E_n(\mathbf{w}) := \frac{1}{2}(t_n - \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_n)^2$, then after presentation of the nth training datapoint, the stochastic gradient descent algorithm updates the parameter vector $\mathbf{w}$ using

$$\boldsymbol{w}^{(\tau)} := \boldsymbol{w}^{(\tau-1)} - \eta^{(\tau)}\nabla E_n(\boldsymbol{w}^{(\tau-1)})$$

where τ denotes the iteration number, and $\eta^{(\tau)}$ is a *learning rate* parameter. We shall discuss the choice of value for $\eta^{(\tau)}$ shortly. The value of $\mathbf{w}$ is initialised to some starting vector $\mathbf{w}^{(0)}$. To summarise, the algorithm is as follows:

- Initialise the parameters to $\mathbf{w}^{(0)}$ and $\tau = 1$
- while a stopping condition is *not* met do:
  - *randomly* select a datapoint $(\mathbf{x_n}, y_n)$ from the training set
  - $\mathbf{w}^{(\tau)} := \mathbf{w}^{(\tau-1)} - \eta^{(\tau)}\nabla E_n(\mathbf{w}^{(\tau-1)})$
  - $\tau = \tau + 1$

Note that τ refers to visiting a *single* training datapoint in Stochastic GD; this is different from t in Batch GD which refers to visiting *all* data points in the training set. For the case of the sum-of-squares error function, the stochastic gradient descent algorithm gives

$$\mathbf{w}^{(\tau)} := \boldsymbol{w}^{(\tau-1)} + \eta^{(\tau)}(t_n - \mathbf{w}^{(\tau-1)}\cdot\boldsymbol{\phi}(\mathbf{x}_n))\boldsymbol{\phi}(\mathbf{x}_n)$$

This is known as the *least-mean-squares* (LMS) algorithm. The value of $\eta^{(\tau)}$ needs to be chosen with care to ensure that the algorithm converges; for example, it can be a decreasing function of the iteration number e.g. $\eta^{(\tau)} = \frac{a}{\tau+b}$ where a and b are positive numbers.

3
Activity 2.1 Linear Regression

In this activity we implement Linear Regression algorithm. This activity helps you to complete Assignment 2.

Please download this (https://www.alexandriarepository.org/wp-content/uploads/20160701172848/Activity2.zip) zip file (right click and then choose "save link as") that contains the dataset (.csv), R script (.r), Jupyter notebook (.ipynb) and output file (.html). If you have access to a Jupyter instance, the notebook (and dataset) is enough to run the experiments. Otherwise, you may read the instructions and discussions from the HTML file and execute the R script

4
Regularisation

In Module 1 (https://www.alexandriarepository.org/reader-preview/84659/91098), we introduced the idea of adding a regularization term to an error function in order to control over-fitting, so that the total error function to be minimized takes the form

$$E_{\mathcal{D}}(\boldsymbol{w}) + \lambda\Omega(\boldsymbol{w})$$

where λ is the regularisation parameter that controls the relative importance of the data-dependent error $E_{\mathcal{D}}$ and the regularisation function Ω. The regularisation function captures the complexity of any model in our model class from which we are searching for the best model.

Regularisation allows complex models to be trained on data sets of limited size without severe over-fitting, essentially by limiting the *effective* model complexity. However, the problem of determining the optimal model complexity is then shifted from one of finding the appropriate number of basis functions to one of determining a suitable value of the regularisation parameter λ .

A suitable definition of the regularisation function Ω depends on the problem at hand. For the space of linear basis functions, a general definition is as follows:

$$\Omega(\boldsymbol{w}) := \frac{1}{2}\sum_{j=0}^{M-1} |w_j|^q$$

As we have seen before, the data dependent error term for regression is usually the sum of squares error function

$$E_{\mathcal{D}} := \frac{1}{2}\sum_{n=1}^{N} (t_n - \boldsymbol{w}\cdot\boldsymbol{\phi}(\boldsymbol{x}_n))^2$$

which is also rooted in maximum likelihood. Putting the error function and the regularisation term together, the resulting training objective is:

$$\frac{1}{2}\sum_{n=1}^{N} (t_n - \boldsymbol{w}\cdot\boldsymbol{\phi}(\boldsymbol{x}_n))^2 + \frac{\lambda}{2}\sum_{j=0}^{M-1} |w_j|^q$$

In what follows, we discuss a few popular choices for q in the regularisation function, and the algorithms to solve the resulting training objectives.

Ridge Regression: L2 Regularisation

Ridge regression corresponds to the case where $q = 2$. The resulting training objective is:

$$\frac{1}{2}\sum_{n=1}^{N}(t_n - \boldsymbol{w}\cdot\boldsymbol{\phi}(\boldsymbol{x}_n))^2 + \frac{\lambda}{2}\sum_{j=0}^{M-1} w_j^2$$
$$= \frac{1}{2}\sum_{n=1}^{N}(t_n - \boldsymbol{w}\cdot\boldsymbol{\phi}(\boldsymbol{x}_n))^2 + \frac{\lambda}{2}\boldsymbol{w}^T\cdot\boldsymbol{w}$$

This particular choice of regulariser is known in the machine learning literature as weight decay because in sequential learning algorithms, it encourages weight values to decay towards zero, unless supported by the data (i.e., only big enough data should result to more complex model).

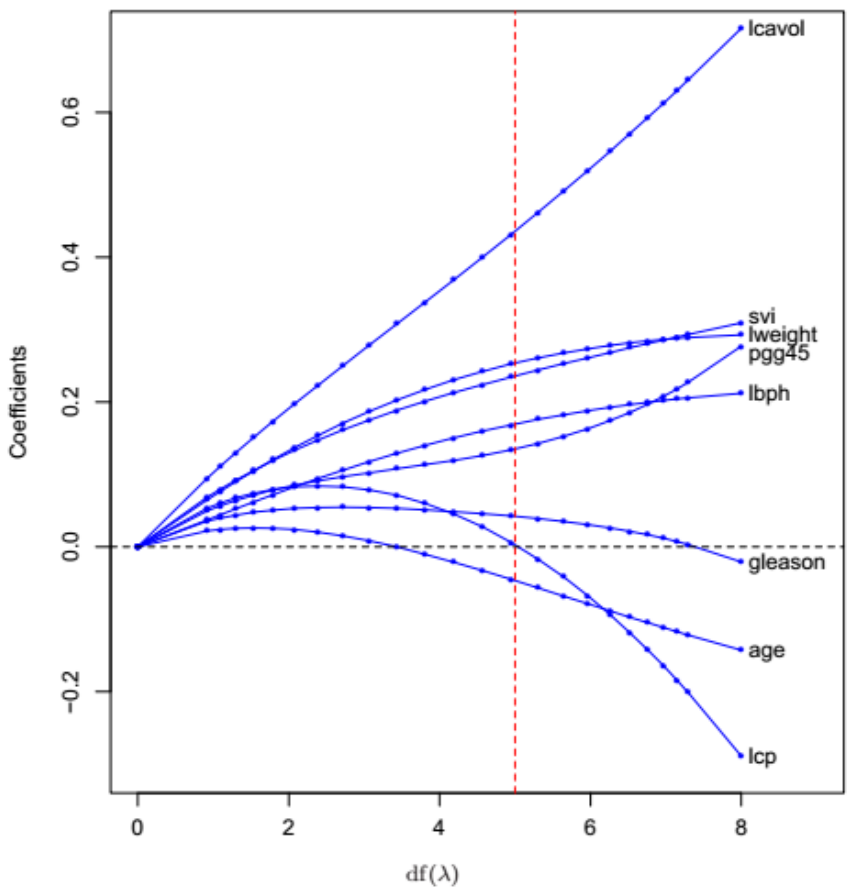

Figure 2.3.1. Ridge coefficients vs the effective degrees of freedom of λ for the prostate cancer. The vertical line indicates df = 5.0 (chosen by cross-validation). Source: ESLII

Finding the Optimal Solution. We can solve ridge regression's training objective with matrix operations. Setting the gradient of the training objective with respect to parameters to zero, and solving for $\boldsymbol{w}$ as before, we obtain

$$\boldsymbol{w} = \left(\lambda\boldsymbol{I} + \boldsymbol{\Phi}^T\boldsymbol{\Phi}\right)^{-1}\boldsymbol{\Phi}^T\mathbf{t}$$

where $\boldsymbol{I}$ is the identity matrix, an M-by-M matrix where the diagonal elements are one and off diagonal elements are zero. This solution represents a simple extension of the least-squares solution that we saw in the previous chapter, where the only difference is adding λ to the main diagonal of $\boldsymbol{\Phi}^T\boldsymbol{\Phi}$

before taking the inverse.

From linear algebra perspective, adding λ to the main diagonal has a very good reason: it makes the resulting matrix to behave well (technically *well-conditioned*) for taking the inverse. For example, even if $\boldsymbol{\Phi}^T\boldsymbol{\Phi}$ is not invertible (due to having zero eigenvalues), adding λ will remove all zero eigenvalues and makes the matrix invertible hence the optimum solution becomes well defined.

LASSO: L1 Regularisation

The case of $q = 1$ is know as the lasso in the statistics literature:

$$\frac{1}{2}\sum_{n=1}^{N}(t_n - \boldsymbol{w}\cdot\boldsymbol{\phi}(\boldsymbol{x}_n))^2 + \frac{\lambda}{2}\sum_{j=0}^{M-1}|w_j|$$

It has the property that if λ is sufficiently large, some of the coefficients w_j are driven to zero, leading to a *sparse* model in which the corresponding basis functions play no role.

Sparsity is an important concept in machine learning and an active area of research. Sparse models are attractive because (1) they are fast in prediction time due skipping the computation of basis functions corresponding to zero parameters, and (2) they lead to interpretable models by choosing - among all basis functions - only a handful of important ones having predictive power (the ones corresponding to non-zero parameters) for the problem at hand.

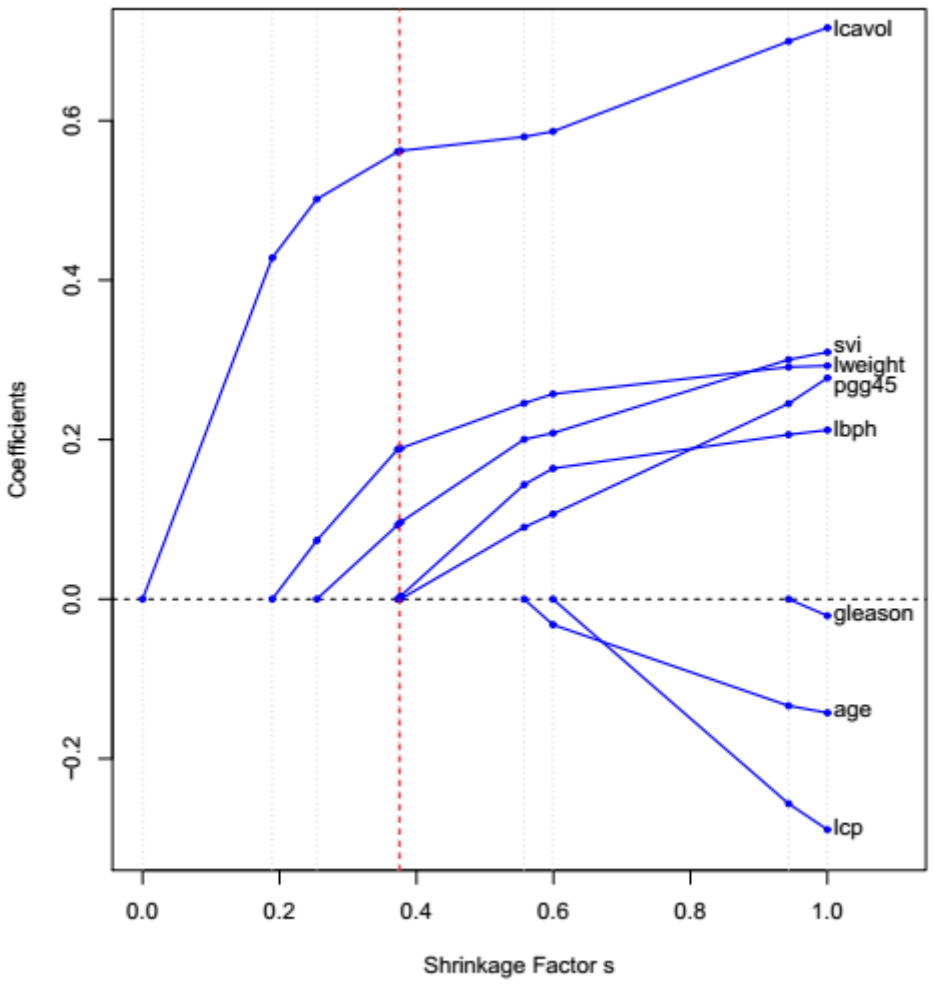

Figure 2.3.2. LASSO coefficients vs shrinking factor (s = t/ Pp 1 |β ˆj|). The vertical line indicates s = 0.36 (chosen by cross-validation). Comparing with Figure 2.3.1, the lasso profiles hit zero, while those for ridge do not. Source: ESLII

Finding the Optimal Solution. We note that the L1-regularisation term is *not* differentiable. Therefore, the training objective is not differentiable, hence it cannot be optimised using the (stochastic) gradient descent algorithm. There are optimisation algorithms though which can be used to optimise non-differentiable functions, such as algorithms based on the *sub-gradient*.

5
Activity 2.2 Regularization

LASSO, Ridge and Elastic Regression

In this activity we study the effect og regularization on the model complexity and generalization. This activity helps you to complete Assignment 2.

Please download this (https://www.alexandriarepository.org/wp-content/uploads/20160701172848/Activity2.zip) zip file (right click and then choose "save link as") that contains the dataset (.csv), R script (.r), Jupyter notebook (.ipynb) and output file (.html). If you have access to a Jupyter instance, the notebook (and dataset) is enough to run the experiments. Otherwise, you may read the instructions and discussions from the HTML file and execute the R script.

For detailed discussion about setting your programming environment up, please refer to Appendix B: Setting up Your Programming Environme (https://www.alexandriarepository.org/?post_type=module&p=97654)

6
Bias-Variance Analysis

So far in our discussion of linear models for regression, we have assumed that the form and number of basis functions are both fixed. As we have seen in Module 1 (https://www.alexandriarepository.org/reader-preview/84659/91098), the use of maximum likelihood, or equivalently least squares, can lead to severe over-fitting if complex models are trained using data sets of limited size. However, limiting the number of basis functions in order to avoid over-fitting has the side effect of limiting the flexibility of the model to capture interesting and important trends in the data. Although the introduction of regularisation terms can control over-fitting for models with many parameters, this raises the question of how to determine a suitable value for the regularisation parameter λ. Seeking the solution that minimises the regularised error function with respect to both the parameter vector $\boldsymbol{w}$ and the regularisation coefficient λ is clearly not the right approach since this leads to the unregularised solution with $\lambda = 0$.

In this chapter, we consider a *frequentist* viewpoint of the model complexity issue, known as the *bias-variance trade-off.* Intuitively, *bias* is a learner's tendency to consistently learn the same wrong thing. *Variance* is the tendency to learn random things irrespective of the real signal. Figure **2.6.1** illustrates this by an analogy with throwing darts at a board. For example, a degree-1 polynomial high bias for regression, because when the underlying unknown function is non-linear, the learner is unable to induce it. High-degree polynomials don't have this problem because they can represent any function reasonably well, but on the other hand they can suffer from high variance: high-degree polynomials learned on different training sets generated by the same phenomenon are often very different, when in fact they should be the same.

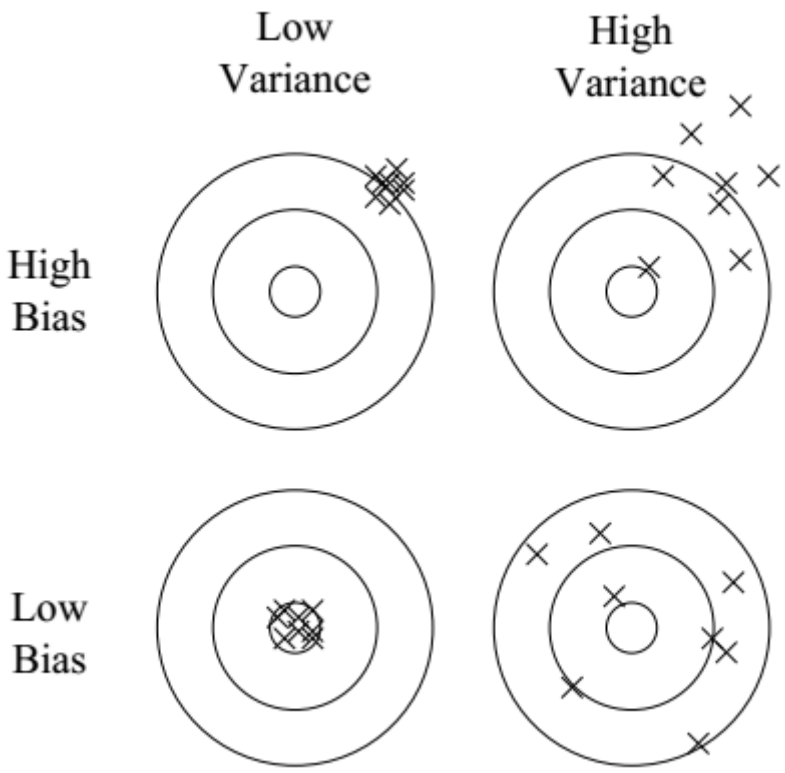

Figure 2.6.1. Bias and variance in dart-throwing. Source: "A Few Useful Things to Know about Machine Learning",Pedro Domingos

In what follows, we formalise the above intuitive inter-play between model complexity (or flexibility) and bias-variance. Although we shall introduce this concept in the context of linear basis function models, where it is easy to illustrate the ideas using simple examples, the discussion has more general applicability.

Bias-Variance Decomposition

The Setup. We are given a training dataset $\mathcal{D} := \{(\boldsymbol{x}_n, t_n)\}_{n=1}^N$ where input data points $\boldsymbol{x}$ are generated according to the distribution $p(\boldsymbol{x})$, and their corresponding target values t are generated according to the function $h(\boldsymbol{x})$. It is important to note that in the training time, we are not given the distribution $p(\cdot)$ and the underlying function $h(\cdot)$ which maps the input to the target. All we are given is the training dataset $\mathcal{D}$ and our goal is to find out the unknown underlying function $h(\cdot)$.

Generalisation Error. Since we don't know the the true underlying function $h(\cdot)$, we are not sure whether it belongs to the class of linear basis function models from which we try to learn a model. Hence, we modify our desire and aim to find a linear basis function model $y(\cdot)$ in our model class which is closest to $h(\cdot)$; in other words, a model which has the smallest *generalisation error:*

$$\mathcal{E}_{h,p}(y) := \int [y(\boldsymbol{x}) - h(\boldsymbol{x})]^2 p(\boldsymbol{x}) d\boldsymbol{x}$$

Again, we stress that the above generalisation error cannot be computed for a candidate model $y(\cdot)$ because we neither have the input distribution $p(\cdot)$ nor the underlying function $h(\cdot)$.

Bootstrap Idea. If we model the $h(\boldsymbol{x})$ using a parametric function $y(\boldsymbol{x}, \boldsymbol{w})$ governed by a parameter vector $\boldsymbol{w}$, then from a Bayesian perspective the uncertainty in our model is expressed through a posterior distribution over $\boldsymbol{w}$. A frequentist treatment, however, involves making a point estimate of $\boldsymbol{w}$ based on the data set $\mathcal{D}$, and tries instead to interpret the uncertainty of this estimate through the following thought experiment. Suppose we had a large number of data sets each of size N and each drawn independently from the distribution $p(\cdot)$. For any given data set $\mathcal{D}$, we can run our learning algorithm and obtain a prediction function $y(\boldsymbol{x}, \mathcal{D})$. Different data sets from the ensemble will give different functions and consequently different values of the squared error. The performance of a particular learning algorithm is then assessed by taking the average over this ensemble of data sets.

Bias-Variance Decomposition. Consider the definition of the above generalisation error $\mathcal{E}$ and the term in the integration, which for a particular data set $\mathcal{D}$ takes the form

$$\{y(\boldsymbol{x}; \mathcal{D}) - h(\boldsymbol{x})\}^2$$

Because this quantity will be dependent on the particular data set $\mathcal{D}$, we take its average over the ensemble of data sets, which we denote by $\mathbb{E}_{\mathcal{D}}[y(\boldsymbol{x}; \mathcal{D})]$. If we add and subtract the quantity

$\mathbb{E}_{\mathcal{D}}[y(\boldsymbol{x};\mathcal{D})]$ inside the braces, and then expand, we obtain

$$\begin{aligned}&\{y(\boldsymbol{x};\mathcal{D})-\mathbb{E}_{\mathcal{D}}[y(\boldsymbol{x};\mathcal{D})]+\mathbb{E}_{\mathcal{D}}[y(\boldsymbol{x};\mathcal{D})]-h(\boldsymbol{x})\}^2\\&=\{y(\boldsymbol{x};\mathcal{D})-\mathbb{E}_{\mathcal{D}}[y(\boldsymbol{x};\mathcal{D})]\}^2+\{\mathbb{E}_{\mathcal{D}}[y(\boldsymbol{x};\mathcal{D})]-h(\boldsymbol{x})\}^2\\&+2\{y(\boldsymbol{x};\mathcal{D})-\mathbb{E}_{\mathcal{D}}[y(\boldsymbol{x};\mathcal{D})]\}\{\mathbb{E}_{\mathcal{D}}[y(\boldsymbol{x};\mathcal{D})]-h(\boldsymbol{x})\}\end{aligned}$$

We now take the expectation of this expression with respect to $\mathcal{D}$ and note that the final term will vanish, giving

$$\begin{aligned}&\mathbb{E}_{\mathcal{D}}[\{y(\boldsymbol{x};\mathcal{D})-h(\boldsymbol{x})\}^2]=\\&\underbrace{\{\mathbb{E}_{\mathcal{D}}[y(\boldsymbol{x};\mathcal{D})]-h(\boldsymbol{x})\}^2}_{\text{bias}^2}+\underbrace{\mathbb{E}_{\mathcal{D}}\left[\{y(\boldsymbol{x};\mathcal{D})-\mathbb{E}_{\mathcal{D}}[y(\boldsymbol{x};\mathcal{D})]\}^2\right]}_{\text{variance}}\end{aligned}$$

We see that the expected squared difference between $y(\boldsymbol{x};\mathcal{D})$ and the regression function $h(\boldsymbol{x})$ can be expressed as the sum of two terms. The first term, called the squared bias, represents the extent to which the average prediction over all data sets differs from the desired regression function. The second term, called the variance, measures the extent to which the solutions for individual data sets vary around their average, and hence this measures the extent to which the function $y(\boldsymbol{x};\mathcal{D})$ is sensitive to the particular choice of data set. We shall provide some intuition to support these definitions shortly when we consider a simple example.

So far, we have considered a single input value $\boldsymbol{x}$. If we substitute this expansion back into the definition of the generalisation error $\mathcal{E}(\cdot)$, we obtain the following decomposition of the expected squared loss:

$$\text{generalisation error}=\text{bias}^2+\text{variance}$$

where

$$\begin{aligned}(\text{bias})^2&:=\int\{\mathbb{E}_{\mathcal{D}}[y(\boldsymbol{x};\mathcal{D})]-h(\boldsymbol{x})\}^2p(\boldsymbol{x})d\boldsymbol{x}\\\text{variance}&:=\int\mathbb{E}_{\mathcal{D}}\left[\{y(\boldsymbol{x};\mathcal{D})-\mathbb{E}_{\mathcal{D}}[y(\boldsymbol{x};\mathcal{D})]\}^2\right]p(\boldsymbol{x})d\boldsymbol{x}\end{aligned}$$

and the bias and variance terms now refer to integrated quantities.

Model Complexity vs Bias-Variance

Our goal is to minimize the generalisation error, which we have decomposed into the sum of a (squared) bias and a variance. As we shall see, there is a trade-off between bias and variance, *with very flexible models having low bias and high variance, and relatively rigid models having high bias and low variance*. The model with the optimal predictive capability is the one that leads to the best balance between bias and variance.

Curve-fitting Example. This is illustrated by considering the sinusoidal data set from Module 1 (https://www.alexandriarepository.org/reader-preview/84659/91098). Here we generate 100 data sets, each containing

$N = 25$ data points, independently from the sinusoidal curve $h(x) = sin(2\pi x)$. The data sets are indexed by $l = 1, \ldots, L$, where $L = 100$, and for each data set $D^{(l)}$ we fit a model with 24 Gaussian basis functions by minimising the regularised error function to give a prediction function $y^{(l)}(x)$ as shown in Figure **2.6.2**.

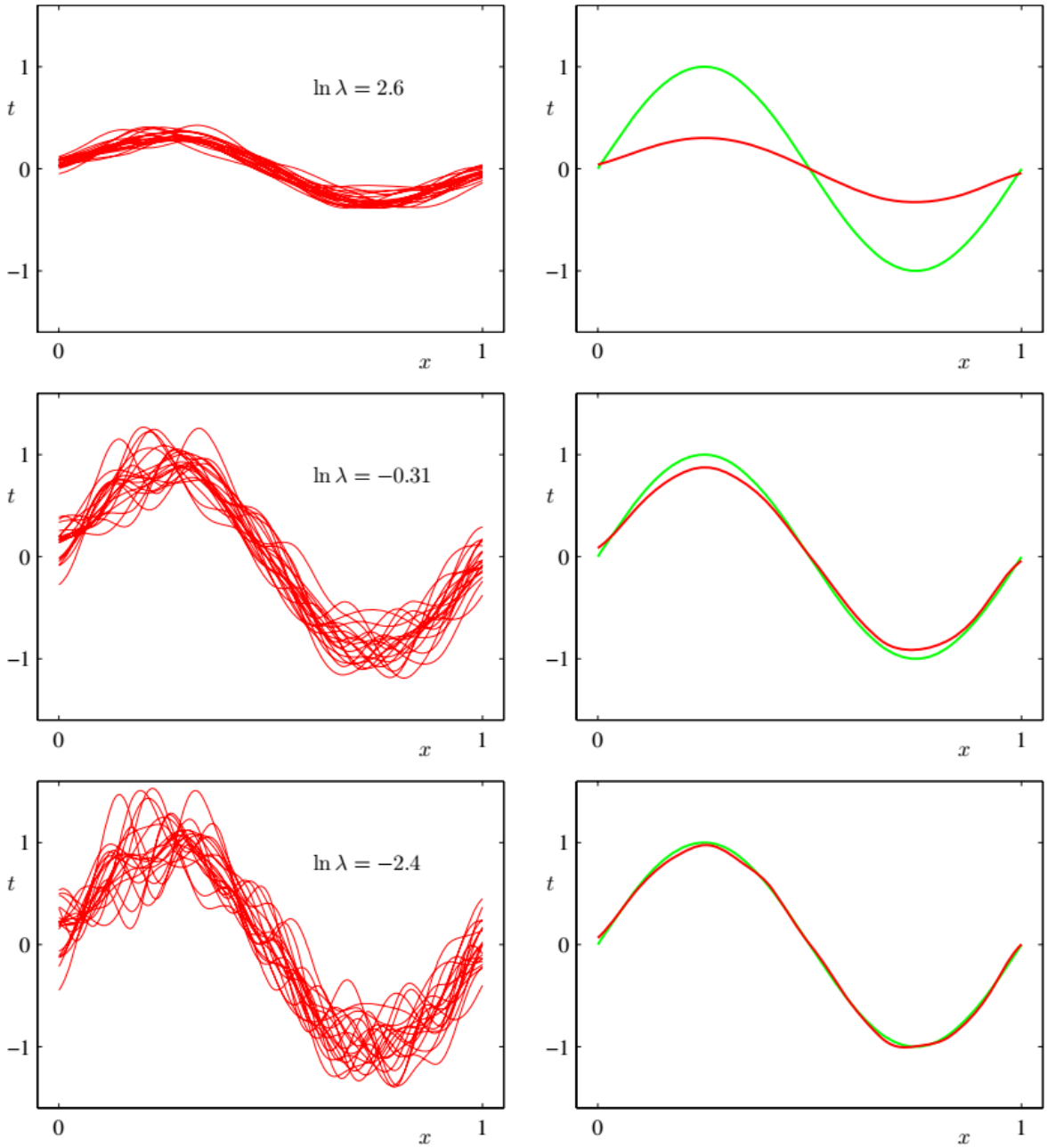

Figure 2.6.2. Illustration of the dependence of bias and variance on model complexity, governed by a regularization parameter λ, using the sinusoidal data set from Chapter 1. There are L = 100 data sets, each having N = 25 data points, and there are 24 Gaussian basis functions in the model so that the total number of parameters is M = 25 including the bias parameter. The left column shows the result of fitting the model to the data sets for various values of ln λ (for clarity, only 20 of the 100 fits are shown). The right column shows the corresponding average of the 100 fits (red) along with the sinusoidal function from which the data sets were generated (green).

The top row in Figure **2.6.2.** corresponds to a large value of the regularisation parameter λ that gives low variance (because the red curves in the left plot look similar) but high bias (because the two curves in the

right plot are very different). Conversely on the bottom row, for which λ is small, there is large variance (shown by the high variability between the red curves in the left plot) but low bias (shown by the good fit between the average model fit and the original sinusoidal function). Note that the result of averaging many solutions for the complex model with $M = 25$ is a very good fit to the regression function, which suggests that averaging may be a beneficial procedure. Indeed, a weighted averaging of multiple solutions lies at the heart of a Bayesian approach, although the averaging is with respect to the posterior distribution of parameters, not with respect to multiple data sets.

Learning Curves. We can also examine the bias-variance trade-off quantitatively for this example. The average prediction is estimated from

$$\bar{y}(x) := \frac{1}{L}\sum_{l=1}^{L} y^{(l)}(x)$$

and the integrated squared bias and integrated variance are then given by

$$\begin{aligned}\text{bias}^2 &:= \frac{1}{N}\sum_{n=1}^{N}\left[\bar{y}(x_n) - h(x_n)\right]^2 \\ \text{variance} &:= \frac{1}{N}\sum_{n=1}^{N}\frac{1}{L}\sum_{l=1}^{L}\left[y^{(l)}(x_n) - \bar{y}(x_n)\right]^2\end{aligned}$$

where the integral over $\mathbf{x}$ weighted by the distribution $p(\mathbf{x})$ is approximated by a finite sum over data points drawn from that distribution. The (average) test error is estimated as

$$\text{test error} := \frac{1}{L}\sum_{l=1}^{L}\frac{1}{N}\sum_{n=1}^{N}\left[y^{(l)}(x_n) - h(x_n)\right]^2$$

These quantities are plotted as a function of $\ln \lambda$ in Figure **2.6.3.** We see that small values of λ allow the model to become finely tuned to the noise on each individual data set leading to large variance. Conversely, a large value of λ pulls the weight parameters towards zero leading to large bias.

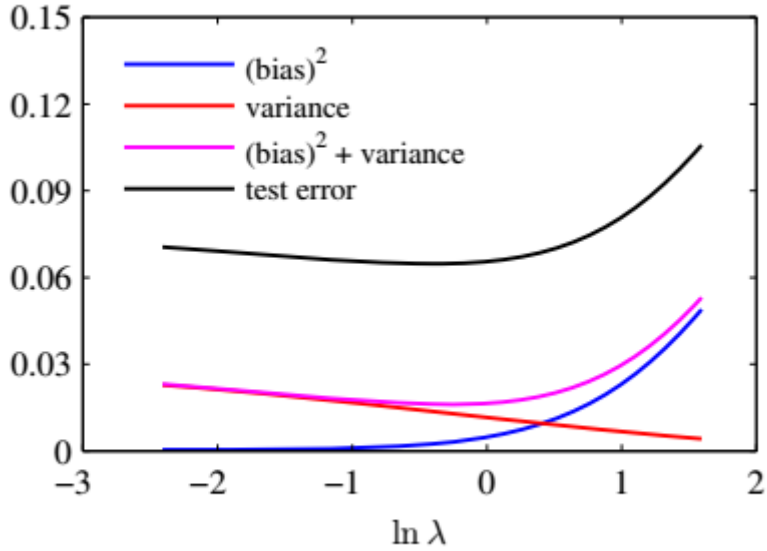

Figure 2.6.3. Plot of squared bias and variance, together with their sum, corresponding to the results shown in Figure 4.2. Also shown is the average test set error for a test data set size of 1000 points. The minimum value of $(\text{bias})^2$ + variance occurs around $\ln \lambda = -0.31$, which is close to the value that gives the minimum error on the test data.

7
Activity 2.3 Bias-Variance in Regression

In this activity we perform Bias-Variance analysis in a regression setting.

Please download this (https://www.alexandriarepository.org/wp-content/uploads/20180810093029/Activity.2.3.ipynb) Jupyter file (right click and then choose "save link as").

For detailed discussion about setting your programming environment up, please refer to Appendix B: Setting up Your Programming Environment (https://www.alexandriarepository.org/?post_type=module&p=97654)